# Failure-Governed Method Evolution without Self-Promotion

Working framework draft

August 2026

**Abstract**

Self-improving research systems face an authority problem: the same process that proposes a method change can often influence the evidence or evaluator used to justify that change. This paper treats method evolution as protected research rather than as direct self-editing, and holds the authority to propose a change apart from the authority to conclude that it is better. Failed, blocked, partial and undetermined executions are retained as persistent evidence; recurrence is separated from causal responsibility; unresolved issues carry competing cause hypotheses and discriminator evidence across interventions; invention is gated until ordinary causes and transfer alternatives have been challenged; candidate changes execute in isolated, content-addressed environments; and method reuse requires replay, fresh transfer and protected assurance bound to the exact candidate and evidence lineage. The controller has no self-merge primitive and its strongest internal outcome is a recommendation to an external host. Local hostile tests exercise recurrence-not-cause, negative-history retention, replay/fresh-transfer separation, invention readiness and protected-path reward-hacking attacks. These results establish implementation and governance semantics only. A diagnostic glm-5.2 hidden-cause attribution archive scores 21/24 with three residual errors retained; this does not establish transferable self-improvement. Whether governed method evolution improves fresh development outcomes remains undetermined: the prospective campaign that would decide it has not been executed, and an outcome that remains undetermined is reported separately from an outcome determined to be false.

**Keywords:** self-improving agents; failure attribution; fresh transfer; evaluator integrity; program evolution; AI governance.

## 1 Introduction

A method-evolving research system must answer two different questions: what change should be tried, and who is allowed to conclude that the change is better. Collapsing those questions creates a self-promotion channel. A coding agent may produce an impressive patch, but if it can also rewrite the benchmark, choose only favorable episodes, change protected governance, or certify its own evidence, improvement is not established.

Development is therefore treated here as an evidence-governed research process rather than as a direct self-edit loop. The claim under test is deliberately narrower than "autonomous self-improvement":

> Failures may become method knowledge only after persistent recording, causal discrimination, nearest-work/transfer challenge, isolated challenger execution, replay, fresh transfer and protected assurance; the candidate process cannot grant itself method authority or merge its own change.

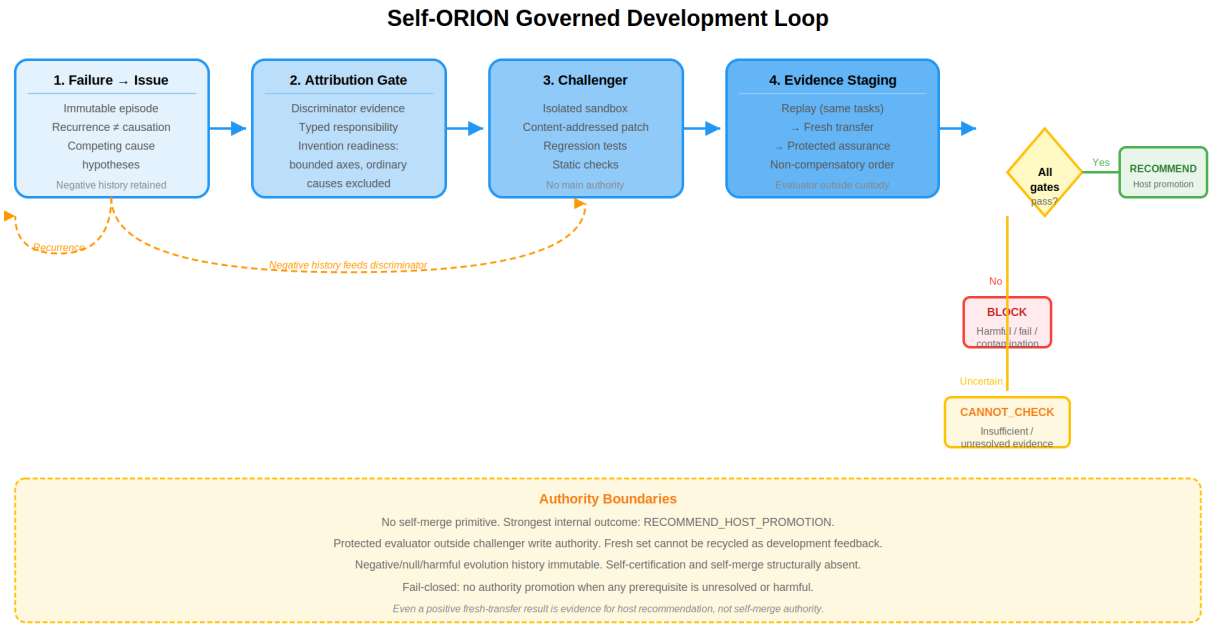


**Figure 1:** The governed development loop. The failure-to-method path proceeds through persistent issue recording, causal discrimination, invention-readiness gating, isolated challenger execution, non-compensatory evidence staging (replay → fresh transfer → protected assurance), and a fail-closed authority boundary. The strongest internal outcome is a host recommendation; self-merge and self-certification are structurally absent.

A method state is not writable merely because a research run failed. A failure first changes the evidence available *about* the method. Only a separately governed transition can change which method is authorized.

## 1.1 Contributions

Four mechanisms carry the argument, and each is a property of the mechanism rather than of any particular system that implements it:

1. a failure-to-method path that separates observation, recurrence, attribution, lesson/challenger and authority;
2. non-self-promotional evolution, in which candidate generation and candidate authorization remain distinct;
3. protected fresh assurance, which requires evidence beyond the episodes used to construct the repair;
4. a separate authority layer whose final promotion remains in external host custody.

These are candidate differences from existing work, not established novelty or empirical superiority. Section 7 places each among its neighbours, and Section 10 states what the present evidence does and does not support.

## 2 Failure and null outcomes as persistent knowledge

Failed, blocked, partial and undetermined executions are stored as addressable experience rather than discarded as execution noise. A failure episode distinguishes the observed mode, propagated effect, task/variation identity, competing causes, detection evidence, recovery actions, residuals and falsifiers. Negative outcomes remain in history after later success.

This makes failure a potential research input, but not an automatic method lesson. Let $e_j$ denote an immutable episode and $\phi(e_j)$ its observed failure signature. Repeated signatures can justify a recurrence hypothesis,

$$\phi(e_1) \approx \phi(e_2) \approx \cdots \approx \phi(e_n),$$

without justifying the causal statement that one mechanism caused every failure. Recurrence answers "does something structurally similar persist?"; attribution answers "which intervention should change the outcome?"

A valid development record therefore keeps failures, nulls and harmful interventions visible. Otherwise an adaptive system can manufacture an apparently improving history by retaining only successful challengers. Negative-history completeness is an assurance coordinate rather than an optional debugging convenience.

Where the record begins matters as much as what it holds. An execution record can expose a missing or failed coordinate of the method it was produced under; interpreting that coordinate as evidence *about* the method is a separate step, and it is deliberately non-automatic, because a missing specification, a provider outage, a data defect and a genuine operator defect call for different responses.

## 3 Causal discrimination and persistent issue state

A development issue persists across multiple investigations and interventions. A versioned development-issue record keeps a stable issue identity together with candidate causes, supported causes, discriminator evidence, failure episodes, intervention outcomes and lifecycle transitions. This prevents each repair attempt from becoming a new context that forgets why earlier alternatives failed.

For issue $u$, let

$$H(u) = \{h_1, h_2, \ldots, h_k\}$$

be competing cause hypotheses. A repair is not licensed merely because one hypothesis is narratively plausible. It is licensed only by discriminator evidence capable of separating the relevant alternatives. Where the discriminator cannot distinguish the hypotheses, the correct state is unresolved rather than an invented causal label.

This ordering is important:

$$\text{observed failure} \rightarrow \text{competing causes} \rightarrow \text{discriminator} \rightarrow \text{attribution} \rightarrow \text{candidate intervention}.$$

Moving the intervention before the discriminator encourages post-hoc explanations of whatever the candidate happened to change.

Persistent issue state also makes negative interventions scientifically useful. A harmful or null repair can narrow the supported cause set, expose a missing interaction, or invalidate an earlier lesson. Such episodes remain attached to the issue rather than being deleted once a later challenger succeeds.

# 4 Nearest-work challenge, challenger archives, and invention readiness

Repeated failure is not read as an instruction to invent a new operator. Before structural invention, ordinary causes, known mechanisms, transfer candidates and mature parent-domain treatments are searched for first. The invention-readiness gate asks whether relevant search axes are bounded-flat, ordinary causes have been excluded to the supported extent, the residual survives distinct variations, and discriminator evidence identifies a missing-structure class rather than a generic score deficit.

Candidate method changes are retained as challengers rather than silently overwriting the incumbent. The archive records candidate identity, generating issue/evidence, attempted intervention, execution result, regression result, fresh-transfer result, protected-assurance outcome and final host disposition. Failed and harmful candidates remain queryable.

This structure absorbs useful mechanisms from evolutionary and self-improving-agent work without inheriting self-authorization. Archives, iterative candidate generation, population search and evaluator-guided optimization may all be valuable proposal mechanisms. They become method authority only through the separate assurance path.

## 4.1 Method-level challengers and method-space expansion

A method challenger is a candidate structural change linked to documented failure or obstruction records, not a claim that a new method has been invented. The versioned method-challenger record binds the subject revision, generating failures, competing ordinary causes already challenged, known-method search routes and their inadequacy, assimilated donor mechanisms, the structural edit and its predeclared discriminator. Candidate generation is therefore separate from host adoption. A candidate must traverse

$$\textsc{Diagnose} \to \textsc{Search/Absorb} \to \textsc{Challenger} \to \textsc{IsolatedRun} \\ \to \textsc{Replay} \to \textsc{Fresh} \to \textsc{Protected} \to \textsc{HostDisposition}.$$

A replay improvement cannot compensate for harmful fresh transfer, and missing protected evidence blocks disposition. Rejected, null and harmful histories remain append-only and queryable; neither a generator nor a challenger can rewrite the evaluator or authority registry. For example, a structural adaptation that improves its motivating replay but harms a protected family on fresh transfer remains rejected, even if its development score is higher. Known-method search and transfer must be challenged before an invention gap is declared; repeated failure alone never makes a method ready for invention. The host alone may adopt or recommend adoption.

This is a governance boundary, not an invention result. The mechanism can govern whether a method candidate is safely evaluated and presented to a host; whether a learned system can generate valuable new method structures is a different question, and nothing here bears on it.

## 4.2 Transferred design knowledge

A prior specification audit re-examined the mature method surfaces this design inherits, treating them as prior design knowledge rather than as results. Fourteen provisional structural dimensions across 59 registered mechanics produced 826 explicit transfer questions; evidence-bound answer records moved that registered structural frontier from 826 to 0 with attributable answer and audit closure.

This result is deliberately narrow:

$$\text{specification closure} \neq \text{empirical validity} \neq \text{promotion authority}.$$

It can reduce rediscovery of already specified mechanics and expose remaining empirical work, but it cannot establish that the transferred contracts improve behaviour on fresh tasks.

## 5 Isolated challenger execution and protected change control

The development controller composes a governed path of the form

$$\begin{aligned}&\text{persistent issue/evidence} \to \text{research and nearest-work challenge} \to \text{candidate proposal}\\ &\to \text{content-addressed patch} \to \text{isolated sandbox execution} \to \text{regression/fresh assurance}\\ &\to \text{append-only evolution record} \to \text{host recommendation.}\end{aligned}$$

A language model, coding agent, program synthesizer or other worker may generate the proposed change. It is not the controller and does not own the evaluator. The proposed bytes are content-addressed before execution so the evaluated candidate is the candidate later presented for promotion.

Protected evaluator, registry and governance paths are hard rejection coordinates. A candidate that improves visible scores while modifying its own protected evaluation surface is not treated as an improved method. Candidate execution occurs behind a host-owned isolation boundary, and the change controller exposes no merge primitive.

The strongest internally representable successful outcome is therefore a recommendation that the host promote the candidate, not "self-merged" or "governed by self declaration." This is a structural authority boundary: even a perfect candidate score cannot compensate for missing or compromised change-control evidence.

## 6 Replay, fresh transfer, and protected assurance

A challenger can overfit the exact failure episodes used to construct it. Three evidence roles are therefore kept separate.

**Replay.** The candidate must address the supporting failure under a bound replay rather than merely changing a proxy score.

**Fresh transfer.** The candidate must be evaluated on distinct task, split, run/evaluation and variation identities not used as the repair's support set. Improvement on visible development evidence combined with regression on fresh assurance is represented explicitly as meta-overfitting rather than averaged into a favorable mean.

**Protected assurance.** The authority-bearing evaluator is outside candidate write authority and binds the exact candidate, subject revision, evidence artifact, evaluator identity, lineage, epoch and split. Self-verification, post-hoc evaluator evidence, non-fresh evidence, mixed subject/evaluator epochs and unresolved checks block promotion.

The readiness correction is intentionally asymmetric. Even a complete independent set of passing checks can reach only a readiness state that is still pending external attestation; final promotion stays outside the evaluated system's own custody. This prevents a boolean checklist, a candidate-generated receipt, or a self-selected benchmark from manufacturing governed method authority.

Negative history is part of assurance. A promotion packet that omits failed or harmful alternatives can give a false picture of selection pressure and regression risk; therefore the archive and external handoff must preserve those outcomes.

## 7 Related work

The neighbouring literature is strongest exactly where this paper does not compete. Every mechanism discussed below is treated as a component the design may absorb, not as a rival: a stronger proposer, attributor or acceptor makes the authority boundary drawn here more useful, not less.

Archives and self-modification came first. ADAS automates agent-system design through a meta-agent and an archive [4]; Darwin Gödel Machine performs open-ended self-code modification with empirical validation, an archive and sandboxing [14]; and ADIAS makes persistent issue identity, lifecycle, evidence and intervention-outcome history a first-class optimization state [5]. Persistent issue state, an archive of candidates and sandboxed validation are therefore inherited here rather than introduced.

Failure attribution is a second neighbourhood, and a crowded one. SAGE uses multi-hypothesis failure attribution to generate evidence-grounded explanations and routes verified root causes to intervention levels [7]. CausalFlow uses counterfactual intervention to estimate causal responsibility and to generate minimal repairs [2]. Failure-driven self-improvement converts failed computer-use trajectories into diagnosed code-level improvements [11]. Each of these answers *which cause explains the failure*. The question they leave open is what an attributed cause is then permitted to authorize, and on whose evidence that authorization rests.

Self-evolving coding systems press on the same boundary from the engineering side. MOSS rewrites agent source from production-failure evidence, validates candidate images by replay, and places deployment behind user consent and rollback checks [3]. Ratchet manages a self-authored skill library with outcome-driven retirement and bounded capacity while measuring held-out transfer [15], and Verifier-as-Gatekeeper shows that harmful skill admission contaminates later evolution, testing heterogeneous pre-commit gating together with frozen-pool transfer [9]. MEGA accumulates durable optimization assets in a typed wisdom graph and uses controlled evaluation to attribute improvement effects across optimization cycles [6], while RewardHackingAgents makes evaluator tampering and held-out leakage measurable through patch and access telemetry and evaluator locking [1]. A current survey of self-evolving coding agents treats feedback reliability, benchmark overfitting, safety, maintainability, cost and generalization as field-level concerns [16]; the position taken here is that those concerns are to be measured, not cited as motivation.

Acceptance statistics form a third neighbourhood. PACE casts repeated incumbent-versus-candidate commits under adaptive self-evolution as anytime-valid paired sequential testing [10], and SEA admits self-modifications through anytime-valid certificates around a frozen base [8]. An anytime-valid acceptor answers when an observed difference is real under optional stopping. It does not answer which evidence the difference is allowed to be computed over, and it is adopted here as a within-stage device rather than as the acceptance rule itself.

Finally, transfer measurement supplies the outcome discipline. PAST-Bench evaluates whether retained experience causes later-task improvement under matched experience-on/off conditions [12], which motivates pathway-sensitive evaluation rather than treating memory presence as learning. SEVA reports that repeated self-evolution can produce benchmark specialists that improve on one benchmark while regressing on another [13], which is why replay evidence and fresh-transfer evidence are measured and reported separately below rather than averaged.

What remains after all of this is narrower and explicitly compositional. A persistent devel-

opment issue may license a method change only after evidence-bound causal discrimination and invention-readiness checks; acceptance additionally requires non-compensatory staged evidence in the order static checks, replay, independent protected fresh transfer and non-harm, and protected assurance, with immutable retention of negative, null and harmful history and of compromise telemetry, with evaluator and holdout custody outside challenger write authority, and with self-certification and merge authority structurally absent. Promotion remains a host decision. Whether that composition yields more transferable and less compromise-prone improvement than simpler self-edit or acceptance systems is an empirical question; architecture alone cannot settle it, and the evidence reported here does not.

# 8 Methods

The design choices in this section were fixed before any result-bearing campaign. Machine-readable artifacts govern rather than this narrative: a frozen design, a hidden-cause case schema, a replay/freshness and harm-accounting policy, and a staged acceptance policy. Each is archived, and Section 11 names where, together with the preregistration identifier under which the design was frozen.

## 8.1 Three-valued outcomes

Every decision this paper measures is three-valued rather than binary. A governance check may be determined to hold, determined to fail, or left *undetermined*; a candidate repair may be determined to transfer, determined to harm, or left undetermined. An outcome that remains undetermined is recorded and reported separately from an outcome determined to be false, and the two are never merged. The distinction is the point: an unbound execution identity, an absent credential, a protected evaluator that does not yet exist, or a campaign that has not been run all produce undetermined outcomes, and an undetermined outcome is not evidence of absence. Throughout this paper an outcome described as *undetermined* is this third value; it is carried in the evidence record and in every downstream count, and missing protected evidence is never silently resolved to zero or to a negative. The published tables print the same third value under its recorded status name, so that a table still awaiting its campaign cannot be misread as a table reporting that nothing happened.

## 8.2 The implementation under test

The implementation evaluated here is the Self-ORION development controller; it is the artifact under test. The rest of the paper describes the governance mechanism rather than the product, so that each claim reads as a claim about the rule and not about a named system. Source, protocols and evidence archives are listed in Section 11.

## 8.3 Local hostile falsifier suite

The current local suite exercises the governance semantics rather than claiming open-world improvement. Registered cases include recurrence without causality, competing-cause discrimination, negative intervention retention, replay/fresh-transfer separation, evidence-bound reopening, invention-readiness constraints and protected-path reward-hacking attacks. A candidate that modifies protected registry or governance state is rejected even when visible and fresh score deltas are constructed to look perfect.

These tests support the statement that the implementation can represent and refuse the registered failure modes. They do not show that it discovers correct causes or produces beneficial repairs on real development problems.

### 8.4 Prospective external comparison

The outcome-blind design is frozen in advance. The primary hypothesis is that the full governed path produces greater *protected fresh-task improvement* than the strongest matched self-edit or self-evolution baseline on hidden-cause development cases, while the harmful-transfer rate remains within a predeclared two-percentage-point non-inferiority guard. The design target for fresh improvement is an absolute five-percentage-point advantage. These values are prospective margins, not observed effects.

Registered hidden-cause families are retrieval miss, routing or planning miss, implementation bug, environment/dependency/tool failure, evaluator or metric bug, representation gap, measurement or specification gap, and genuine method-basis gap. The candidate sees the symptom and context but not the protected root-cause label. Each episode separates motivating evidence, replay tasks and an independently frozen fresh-transfer set that changes at least one task, domain, model, environment, data or tool axis.

Strong baselines include a fixed agent, direct self-edit, meta-agent design search, archive-based self-edit, issue-centric optimization, multi-hypothesis attribution, counterfactual repair, failure-driven patching and evaluator-only evolutionary search where runnable. Ablations remove persistent issue state, causal discrimination, invention readiness, independent fresh transfer, protected evaluator custody, negative history and access telemetry; a self-certification or self-promotion ablation is permitted only in an isolated benchmark safety lane and cannot affect the production authority path.

Primary outcomes are protected fresh-task improvement, fresh-transfer success and harmful-transfer rate. Secondary outcomes include cause macro-F1, false method-change rate, intervention success conditional on attribution, evaluator, holdout and negative-history compromise, correct blocking of an undecidable case, recognized-failure recurrence, negative-history completeness, motivating-task improvement and resource cost. Wilson intervals are used for standalone rates and paired percentile bootstrap intervals for matched improvement differences; stochastic repetitions are nested within episode and system. Harmful tail and family regressions are reported separately rather than averaged away.

### 8.5 Design freeze and fresh-transfer custody

The design is frozen and the freeze records that no outcome had been accessed when it was cut. This is not yet an execution freeze. The exact subject revision, hidden-cause suite hash, motivating/replay/fresh split hashes, provider and model versions, baseline configurations, evaluator artifact and evaluation epoch all remain unbound. The fresh evaluator and holdout must be frozen outside challenger write authority before candidate generation, and the fresh set cannot be recycled as development feedback without losing its freshness. Search, file, evaluator and patch access is retained for contamination and integrity review. The external transfer benchmark used as a reference point is pinned to a specific upstream revision, recorded with the protocol; any actual use of it still requires a frozen task and split manifest.

### 8.6 Statistical methods

Standalone rates use Wilson 95% intervals. Matched improvement differences, when an archive exists, use paired percentile bootstrap with 10 000 resamples nested in episode × system × seed. The primary hypothesis family is protected fresh-task improvement with a harmful-transfer guard; Holm correction applies to inferential secondary attribution and governance families. Catastrophic fresh regressions are reported by family and are not averaged away. Missing protected evidence remains undetermined and is never entered as zero. The diagnostic 21/24 attribution interval in Table 1 is a standalone Wilson interval on $n = 24$; it is not a primary effect size.

### 8.7 Threat model

Evaluator compromise, holdout leakage, search contamination, replay-only laundering, missing-stage compensation, negative-history deletion and self-certification are treated as first-class failure modes rather than as missing data. A candidate that can rewrite the protected evaluator, recycle the fresh set as development feedback, or certify its own readiness has not produced an admissible improvement. Local hostile tests exercise registered instances of those attacks; unknown gaming strategies remain outside the present evidence.

### 8.8 Prospective successor design

A successor design, implemented but not executed, adds a narrower formal safeguard ahead of any broader self-revision experiment. Competing revision-responsibility hypotheses may remain ambiguous or unresolved, and required interface-adequacy checks may block escalation from an observed failure to a model, method or objective rewrite while a narrower observation, measurement or representation repair remains indicated. The implementation can nominate a claim-relative minimal candidate mechanic for later testing, but the responsibility report, the interface report and the revision gate are all structurally non-authorizing: they do not replace the invention-readiness gate, they do not grant adoption or promotion, and they supply no evidence for the hypotheses above.

A second tranche makes three further distinctions explicit without granting new operating authority. Internal computation is itself represented as a costed epistemic action, so a hard verification obligation can require an expensive check even when a cheaper action has greater scalar expected value, while optional non-positive computation may stop locally. An uncertainty-containment envelope can restrict the contexts in which a model or claim is admissible without rewriting the model or certifying it as correct. And social corroboration is evidence-bound: different worker or verifier identities are not counted as independent when their registered upstream sources or direct observations overlap, and unresolved hidden contributors prevent exclusive causal credit. These are implementation contracts for later countermodels and experiments, not evidence that adaptive computation, containment or social-state tracking improves any outcome.

The successor experiment is a separate, protected-custody revision-level benchmark rather than a rewrite of the frozen design above. Its development-only cause-confusable panel exposes the same candidate-visible symptom under different protected revision responsibilities; it evaluates matched no-revision, broad-self-edit, method-open-only, world-model, representation-regime, generic-diagnosis and fully composed policies, and it includes feedback-permutation and no-feedback controls. A separate scorer evaluates protected revision responsibility, false broad revision, unresolved decisions, and later fresh-transfer and harm records. This establishes only that the experiment can be run and audited. Its confirmatory preflight is undetermined: the final subject, split, evaluator and protected-suite identities are unbound, and the load-bearing donor comparators do

**Table 1:** Hidden-cause attribution confusion matrix from the archived glm-5.2 diagnostic run. Accuracy is 21/24 (Wilson 95% [0.690, 0.957]). Three residual errors are retained and are not relabeled as successes. This is not a protected fresh-transfer or H1 result.

| gold / attr. | Retr | Rout | Impl | Env | Eval | Repr | Meas | Meth |
|---|---|---|---|---|---|---|---|---|
| Retr | 2 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| Rout | 0 | 3 | 0 | 0 | 0 | 0 | 0 | 0 |
| Impl | 0 | 0 | 3 | 0 | 0 | 0 | 0 | 0 |
| Env | 0 | 0 | 1 | 2 | 0 | 0 | 0 | 0 |
| Eval | 0 | 0 | 0 | 0 | 3 | 0 | 0 | 0 |
| Repr | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 1 |
| Meas | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 0 |
| Meth | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 3 |

*Residual errors:* P5-HC-002: gold RETRIEVAL_MISS → REPRESENTATION_GAP (MEDIUM) P5-HC-012: gold ENVIRONMENT_DEPENDENCY_TOOL_FAILURE → IMPLEMENTATION_BUG (HIGH) P5-HC-018: gold REPRESENTATION_GAP → METHOD_BASIS_GAP (HIGH).

not yet have frozen structural bindings. The development panel therefore supports no hypothesis and closes no study.

### 8.9 Authority boundary

Whether governed method evolution improves fresh development outcomes remains undetermined until the execution identities are bound and the prospective observations exist. Repository continuous integration, structural specification closure, local hostile tests, replay-only success, a diagnostic 21/24 attribution score, and an internal readiness state are each incapable of resolving that boundary. Even a positive fresh-transfer result would be evidence for an external host recommendation rather than for self-merge authority.

## 9 Results: the diagnostic attribution archive

The only result-bearing archive in this revision is a single-model hidden-cause attribution run over 24 constructed cases, three in each of the eight registered families, produced with the glm-5.2 model. The metrics below are recomputed from the raw per-case records; the sidecar report is checked against those rows and is refused if it disagrees.

Accuracy is $21/24 = 0.875$, with the Wilson 95% interval given in Table 1. This is a descriptive diagnostic score. It is not protected fresh-task improvement, it is not a matched baseline comparison, and it does not test the primary hypothesis.

Three residual errors are retained and are not relabeled as successes (Table 2). Case P5-HC-002, whose protected cause was a retrieval miss, was attributed to a representation gap at medium confidence; case P5-HC-012, an environment, dependency or tool failure, was attributed to an implementation bug at high confidence; and case P5-HC-018, a representation gap, was attributed to a method-basis gap at high confidence.

Replay-versus-fresh scatter, longitudinal specialist regression, the improvement/integrity frontier, failure recurrence, cost to protected improvement, the matched baseline and ablation comparison, and the campaign record of harmful and null interventions all have no admissible rows in this archive. Each of the seven tables below therefore names the measurement it would carry and leaves its result undetermined; no number in them is imputed from the diagnostic accuracy above.

**Table 2:** Retained residual attribution errors from the archived glm-5.2 diagnostic run. These three cases remain incorrect; they are not successes.

| case | gold cause | attributed cause | confidence |
|---|---|---|---|
| P5-HC-002 | retrieval miss | representation gap | medium |
| P5-HC-012 | environment dependency or tool failure | implementation bug | high |
| P5-HC-018 | representation gap | method basis gap | high |

**Table 3:** Replay-vs-fresh scatter: no admissible rows in the archived glm-5.2 attribution run. The archive contains a single-model hidden-cause diagnostic over 24 cases and no campaign-level replay/fresh score pairs. This table will be populated when a live protected-transfer campaign produces matched replay and fresh-task measurements.

| Round | Replay score | Fresh score |
|---|---|---|
| `CANNOT_CHECK` — no admissible data in archive | | |

*Empirical authority:* DESCRIPTIVE_ONLY (diagnostic attribution). Not a protected fresh-transfer or H1 result.

## 10 Limitations and threats to validity

This revision establishes an implemented governance path, a local hostile falsifier suite, a frozen prospective design, and one diagnostic attribution archive. It does not establish transferable protected improvement, reduced harmful transfer, or an integrity advantage against matched self-edit or self-evolution baselines.

**What is not claimed as novel.** This is not the first self-improving agent, the first evolutionary program-search system, the first candidate archive, the first issue-centric self-improvement state, or the first failure-driven adaptation loop. Meta-agent design search, archive-based self-modification, persistent issue state, multi-hypothesis attribution, counterfactual repair, source-level self-rewriting with replay validation, pre-commit gating, evaluator locking and anytime-valid acceptance are all prior work; they are discussed and cited in Section 7, and each is absorbed rather than claimed. What is left as a candidate difference is the composition: the ordered, non-compensatory sequence from persistent issue state through causal discrimination, invention readiness, isolated execution, replay, independent protected fresh transfer and protected assurance, with promotion authority structurally outside the evaluated system. That is a claim about the safety and reliability of a development process, and architecture alone does not establish it.

**The diagnostic archive does not test the primary hypothesis.** The archive scores 21/24 on hidden-cause labels. That quantity is a single-model, single-run descriptive accuracy with a wide Wilson interval. It does not measure protected fresh-task improvement, harmful transfer, or false protected acceptance. The three residual errors remain errors.

**The 24-case suite is not an execution-frozen campaign.** The suite covers eight families, but evaluator hashes and several fresh-task hashes remain placeholders. Motivating, replay and fresh split identities, provider and model revisions, and the protected evaluator epoch are all unbound in both frozen protocol revisions.

**Table 4:** Longitudinal specialist regression: no admissible rows in the archived glm-5.2 attribution run. The archive contains a single-model hidden-cause diagnostic over 24 cases and no longitudinal specialist-designation data. This table will be populated when a live campaign produces round-by-round specialist scores with regression annotations.

| Round | Specialist | Prior score | Current score |
|---|---|---|---|
| `CANNOT_CHECK` — no admissible data in archive | | | |

*Empirical authority:* DESCRIPTIVE_ONLY (diagnostic attribution). Not a protected fresh-transfer or H1 result.

**Table 5:** Improvement-integrity frontier: no admissible rows in the archived glm-5.2 attribution run. The archive contains a single-model hidden-cause diagnostic over 24 cases and no frontier-pair data. This table will be populated when a live campaign produces matched improvement and integrity scores across rounds.

| Round | Improvement score | Integrity score |
|---|---|---|
| `CANNOT_CHECK` — no admissible data in archive | | |

*Empirical authority:* DESCRIPTIVE_ONLY (diagnostic attribution). Not a protected fresh-transfer or H1 result.

**No matched baselines or ablations were executed.** The registered comparators and governance ablations exist as protocol names. They have no result-bearing archive on this subject, so the matched baseline and ablation comparison remains undetermined rather than null.

**No live development cycle is bound.** The live-trial packet still records an unbound corpus revision. Provider and protected-verifier credentials were unset for this revision, so no live campaign was started. A previously advertised report of a flawless run is not used: its artifact identity and repository identity diverged before integration.

**Statistical threats.** $n = 24$ with one seed cannot support the predeclared five-percentage-point fresh-improvement target or the two-percentage-point harm guard. There is no paired bootstrap against a baseline, no multiplicity-adjusted secondary family, and no tail or family regression from a campaign. The macro-F1 reported in the original sidecar assumed flawless precision and is not promoted.

**Contamination and evaluator custody.** The diagnostic run used locally visible gold labels to score attributions after the fact. That is admissible for an offline label-recovery diagnostic. It is not admissible as protected-evaluator evidence: the candidate-facing study still requires labels, fresh payloads and scoring rubrics to remain outside challenger custody.

## 11 Data and code availability

The implementation repository contains the development controller, the protected assurance path, the local hostile falsifier suite, the hidden-cause case suite, the diagnostic attribution archive and the table generator. Every artifact named in the narrative is listed here with its path, so that no repository location has to be carried in the prose. Source, protocols and evidence are version-controlled in the `SzeChunYiu/ORION` repository, under `papers/paper-05-self-orion/`; persistent DOI assignment remains pending publication.

**Table 6:** Failure recurrence: no admissible rows in the archived glm-5.2 attribution run. The archive contains a single-model hidden-cause diagnostic over 24 cases and no recurrence annotations. This table will be populated when a live campaign records which previously resolved failures recur across rounds.

| Original failure | Round resolved | Round recurred | Outcome |
|---|---|---|---|
| CANNOT_CHECK — no admissible data in archive | | | |

*Empirical authority:* DESCRIPTIVE_ONLY (diagnostic attribution). Not a protected fresh-transfer or H1 result.

**Table 7:** Cost to validated improvement: no admissible rows in the archived glm-5.2 attribution run. The archive contains a single-model hidden-cause diagnostic over 24 cases and no cost-to-improvement annotations. This table will be populated when a live campaign records the resource cost (e.g., API calls, rounds, wall time) required to reach validated improvement.

| Improvement | Validation | Cost metric | Value |
|---|---|---|---|
| CANNOT_CHECK — no admissible data in archive | | | |

*Empirical authority:* DESCRIPTIVE_ONLY (diagnostic attribution). Not a protected fresh-transfer or H1 result.

## Preregistration

The study was designed and frozen before any outcome was accessed, under the identifier `P5.hidden-cause-fresh-transfer.v1`. Its recorded state is a design freeze with no outcome accessed. A reader checking whether a reported analysis was planned or chosen after the fact should compare against that freeze; it is the reason the prospective margins in Section 8 can be read as commitments made before, rather than after, seeing any result.

## Frozen design, case structure and acceptance rules

The design freeze is `protocol/PROTOCOL_V1.json`; the hidden-cause case structure is `protocol/HIDDEN_CAUSE_CASE_SCHEMA_V1.json`; replay/freshness definitions and harm accounting are `protocol/FRESH_TRANSFER_POLICY_V1.md`; and the staged acceptance rules and their successor revision are `protocol/PROTOCOL_V2.json` and `protocol/STAGED_ACCEPTANCE_POLICY_V2.md`. The protected-suite freeze procedure is `protocol/PROTECTED_SUITE_FREEZE_V1.md`, and the retention chain for negative history is `protocol/P5_NEGATIVE_HISTORY_CHAIN_V1.json`. The revision-level successor benchmark of Section 8 is frozen as `protocol/SELF_ORION_V3_REVISION_LEVEL_PROTOCOL_V1.json`. The external transfer benchmark used as a reference point is pinned to `Gen-Verse/PAST-Bench@f8223517ae7491e776b69793d9f11e9d074ab42e`; any use of it still requires a frozen task and split manifest.

The mapping from every result-bearing sentence in this manuscript to the archived artifact that supports it is `evidence/CLAIM_LEDGER_V1.json`, with a human-readable view at `evidence/CLAIM_LEDGER_V1.md`. The local hostile falsifier suite is `evidence/FALSIFIER_V1.md`; the protected hidden-cause suite is `evidence/hidden-cause-suite/PROTECTED_SUITE_V1.json`; and the nearest-work dispositions that Section 7 discusses in prose are recorded per mechanism in `evidence/TABLE_P5_1_NEAREST_WORK.json`.

**Table 8:** Baseline and ablation transfer/integrity results: no admissible rows in the archived glm-5.2 attribution run. The archive contains a single-model hidden-cause diagnostic over 24 cases and no baseline/ablation arm data. This table will be populated when a live campaign produces matched baseline-vs-ablation transfer and integrity measurements.

| Condition | Transfer score | Integrity score | Δ vs. baseline |
|---|---|---|---|
| `CANNOT_CHECK` — no admissible data in archive | | | |

*Empirical authority:* DESCRIPTIVE_ONLY (diagnostic attribution). Not a protected fresh-transfer or H1 result.

**Table 9:** Campaign harmful/null interventions: no admissible rows in the archived glm-5.2 attribution run. The archive contains a single-model hidden-cause diagnostic over 24 cases and no intervention-level outcome annotations. This table will be populated when a live campaign records which interventions were harmful, null, or beneficial relative to the incumbent.

| Intervention | Outcome | Effect size | Recovery |
|---|---|---|---|
| `CANNOT_CHECK` — no admissible data in archive | | | |

*Empirical authority:* DESCRIPTIVE_ONLY (diagnostic attribution). Not a protected fresh-transfer or H1 result.

### Diagnostic attribution archive

The raw per-case records are `evidence/glm-5.2-attribution/results.jsonl`, the checked sidecar report is `evidence/glm-5.2-attribution/report.json`, and the reproduction receipt is `evidence/glm-5.2-attribution/REPRODUCTION_RECEIPT.json`. The generated table data are under `evidence/tables/` and their rendered forms under `manuscript/tables/`. Architecture diagrams and pseudocode for the controller, and the historical technical companions this design inherits from, are retained under `research/technical-companions/`.

## 12 Reproducibility

Regeneration of every archived result, without executing a live system, is

```
make paper05-results
```

which runs `python -m orion.study.p5.tables`. The command recomputes 21/24 from the raw records, writes the confusion matrix and the residual-error ledger, and writes stubs for the seven tables that have no admissible rows. A rewrite of the archive that hides the three residual errors is refused with an error exit; a missing archive is reported as undetermined under its own distinct exit code rather than as a build failure, so that an absent measurement cannot be read as a failed build or as a negative result. Exact commands are in `papers/paper-05-self-orion/REPRODUCE.md`. Continuous integration regenerates these artifacts on a clean checkout and requires every committed file to be byte-identical to what the command produces.

### Compute and conservative-abstention cost

The diagnostic archive records 14 190 tokens and a mean latency of ≈13 s per case, for attribution only. Cost and time to *protected validated improvement* remain undetermined. No campaign-level abstention/cost trade-off is reported.

### Licensing

This revision does not add a repository-level license. Redistribution terms therefore remain undetermined from the repository state until they are declared explicitly.

## 13 Ethics and safety

This work studies governed method evolution for research software. The frozen local falsifier and the diagnostic attribution suite use constructed hidden-cause cases. They do not involve human-subject records or personally identifiable information. No live provider campaign was executed in this revision.

The controller has no self-merge primitive, and its strongest internal terminal is a host-only promotion recommendation. Isolated self-certification ablations, if ever executed, are forbidden on the production authority path. Negative, null, blocked and undetermined outcomes are retained rather than dropped.

## 14 Conclusion

Method evolution is treated here as a research-and-authority problem rather than as an invitation for an agent to edit itself. Failure episodes remain evidence, recurrence remains distinct from cause, issues persist across discriminators and interventions, invention is gated by search and attribution, challengers execute in isolation, replay is separated from fresh transfer, protected assurance sits outside candidate write authority, and final promotion remains in host custody. The successor formalism sharpens the same boundary: an anomaly is not itself permission to revise, an unresolved responsibility is not a licence to choose among incompatible repairs, an interface defect is not evidence that the method basis should be rewritten, more expected computational value is not scientific authority, a validity region is not a truth certificate, and repeated social reports are not independent evidence merely because they came from different processes.

The local hostile programme demonstrates these registered governance semantics and has already exposed authority failures in earlier readiness designs. The diagnostic archive scores 21/24 on hidden-cause labels and retains three residual errors; that descriptive result supports none of the paper's hypotheses. The revision-level successor experiment is implemented but neither confirmatory-frozen nor executed, so it too supplies no positive scientific result. Architecture readiness does not imply that candidate causes are scientifically correct, that the development policy chooses valuable work, that repairs transfer, or that a protected evaluator is sufficient against unknown gaming strategies.

The scientific question that remains is whether this composition improves fresh development outcomes while reducing harmful repair, evaluator gaming, false broad revision and false promotion, relative to matched alternatives. Until a campaign executes under complete protected identities and independent verification, transferable protected fresh-task improvement over matched self-edit and self-evolution baselines remains undetermined — which is a different statement from its having been determined to be false, and it is recorded and reported separately.

## References

[1] Yonas Atinafu and Robin Cohen. Rewardhackingagents: Benchmarking evaluation integrity for llm ml-engineering agents. *arXiv preprint arXiv:2603.11337*, 2026.

[2] Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. Causalflow: Causal attribution and counterfactual repair for llm agent failures. *arXiv preprint arXiv:2605.25338*, 2026.

[3] Qianshu Cai, Yonggang Zhang, Xianzhang Jia, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. Moss: Self-evolution through source-level rewriting in autonomous agent systems. *arXiv preprint arXiv:2605.22794*, 2026.

[4] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

[5] Lekang Jiang, Bohan Tang, Stephan Goetz, and Yiwen Guo. Adias: Automated design of interactive agentic systems. *arXiv preprint arXiv:2608.06410*, 2026.

[6] Jung Hwan Lee, Kyu Ho Lee, and Gwang Hoon Yoo. Mega: Self-evolving agent optimization infrastructure via wisdom graph. *arXiv preprint arXiv:2608.10504*, 2026.

[7] Jie Ma et al. One reflection is not enough: Self-correcting autonomous research via multi-hypothesis failure attribution. *arXiv preprint arXiv:2606.31478*, 2026.

[8] Biswa Sengupta. Self-evolving agents with anytime-valid certificates. *arXiv preprint arXiv:2607.00871*, 2026.

[9] Linfang Shang, Ming Xu, Yiding Sun, Tianle Xia, Lingxiang Hu, Lan Xu, and Ning Zheng. When self-evolution backfires: Pre-commit gating against skill contamination in llm agents. *arXiv preprint arXiv:2608.05810*, 2026.

[10] Zayx Shawn. Pace: Anytime-valid acceptance tests for self-evolving agents. *arXiv preprint arXiv:2606.08106*, 2026.

[11] Xueqiao Sun, Xiaohan Wang, Ludwig Schmidt, Serena Yeung-Levy, and Yuhui Zhang. Learning from failure: Inference-time self-improvement for computer-use agents. *arXiv preprint arXiv:2606.31270*, 2026.

[12] Shuhan Xue et al. Past-bench: Benchmarking the foundations of recursive self-improvement in personal agents. *arXiv preprint arXiv:2608.04003*, 2026.

[13] Aojie Yuan, Yi Nian, Haiyue Zhang, Zijian Su, and Yue Zhao. Seva: Self-evolving verification agent with process reward for fact attribution. *arXiv preprint arXiv:2606.29713*, 2026.

[14] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[15] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Ratchet: A minimal hygiene recipe for self-evolving llm agents. *arXiv preprint arXiv:2605.22148*, 2026.

[16] Hao Zhou, Haichuan Hu, Ye Shang, and Quanjun Zhang. Self-evolving coding agents. *arXiv preprint arXiv:2608.03392*, 2026.